GCP & Vertex AI Features — Complete Research for Agent Hub

> **Date:** March 10, 2026
> **Purpose:** Exhaustive inventory of Vertex AI and Google Cloud Platform features applicable to the Gemini Live Agent Challenge hackathon project.
> **Goal:** Maximize "Google Cloud Native" judging score by using 16+ GCP services meaningfully.

Table of Contents

1. Vertex AI Features — Complete Inventory
    - Tier 1: Core (Must Use)
    - Tier 2: High-Value Additions
    - Tier 3: Impressive But Niche
2. Other GCP Services — Beyond Vertex AI
    - Already Planned
    - Worth Adding
3. Recommended Architecture Stack
4. Feature Deep Dives
    - Vertex AI Agent Engine
    - Agent Engine Sessions
    - Agent Engine Memory Bank
    - Agent Engine Code Execution
    - Gen AI Evaluation Service
    - Grounding with Google Search
    - Grounding with Google Maps
    - RAG Engine
    - Imagen 4
    - Gemini Live API
5. Implementation Priority & Effort Matrix
6. GCP Service Count — Competitive Analysis

---

1. Vertex AI Features — Complete Inventory

Tier 1: Core (Must Use)

These are non-negotiable for our project. They form the product's backbone.

Feature	What It Does	Our Use Case	Judges See
Gemini Live API	Bidi WebSocket streaming — 16kHz PCM in, 24kHz PCM out, VAD, barge-in, native audio	The entire voice pipeline. Model: `gemini-live-2.5-flash-native-audio`	The product's core. Mandatory.
Affective Dialog	Model senses user tone/emotion and adapts response style	Agent sounds empathetic when user is frustrated, energetic when user is excited	Judges hear natural voice — scores "fluidity"
Proactive Audio	Model speaks first when context warrants	Agent notices new file opened on desktop → offers to help unsolicited	"Beyond text" paradigm — agent initiates
Grounding with Google Search	Model calls Google Search to verify facts, returns inline citations + Search Suggestions	Every factual question gets grounded. Anti-hallucination.	Required for "Avoids hallucinations" rubric
ADK (Agent Development Kit)	Multi-agent orchestration, `run_live()`, McpToolset, Callbacks, Sessions, Context Compression	The entire backend brain	"Effective use of ADK" rubric

Key Technical Details:

- Live API model: `gemini-live-2.5-flash-native-audio` (GA, recommended)
- Deprecated model: `gemini-live-2.5-flash-preview-native-audio-09-2025` — removal March 19, 2026
- Audio specs: Input 16-bit PCM 16kHz, Output 16-bit PCM 24kHz, little-endian
- Protocol: Stateful WebSocket (WSS)
- 24 supported languages for multilingual voice

Tier 2: High-Value Additions

These significantly boost judge impression and are worth the implementation effort.

Feature	What It Does	Our Use Case	Effort	Judge Impact
Vertex AI Agent Engine	Managed runtime for deploying ADK agents. Includes Sessions, Memory Bank, Code Execution, Tracing, Monitoring	Deploy to Agent Engine for managed scaling + built-in sessions + observability. OR use Agent Engine Sessions/Memory Bank as services alongside Cloud Run	4-6 hrs	**HUGE** — "Vertex AI Agent Engine" in architecture = deep GCP native
Agent Engine Sessions	Managed stateful session storage for agents	Replace custom Firestore session service. Sessions persist automatically across restarts	2 hrs	Solves biggest technical risk (session persistence) with Google-managed service
Agent Engine Memory Bank	Long-term memory across sessions — stores & retrieves personalized info from past conversations	Agent remembers "last week you asked about Tesla stock" even across new sessions. Personalization layer	3 hrs	**Novel** — most entries won't have long-term memory. Judges notice
Agent Engine Code Execution	Managed sandboxed code execution (alternative to E2B)	Replace or complement E2B — keeps everything on GCP. Fallback if E2B is down	2 hrs	More "Google Cloud Native" points
Gen AI Evaluation Service	Automated quality testing with adaptive rubrics — generates unique pass/fail tests per prompt	Run evaluation suites: "Does Nova answer financial questions accurately?" "Does Atlas write correct code?"	3-4 hrs	Shows **production mindset**. Judges evaluating "Technical Implementation" love eval tests
Grounding with Google Maps	Model accesses Google Maps data for location-based queries	"Where's the nearest coffee shop?" → agent returns real location data. Cross-client: glasses user asks → map card on dashboard	1-2 hrs	Another grounding source = more "Google Cloud" usage
RAG Engine	Managed vector database + retrieval pipeline for document-based Q&A	Upload user documents → agent answers questions about them. Supports Google Drive and GCS	4-6 hrs	Enterprise-grade feature. Heavy GCP native bonus
Imagen 4	Image generation from text prompts (2K resolution)	Agent tool: "Generate an image of a sunset" → image renders as GenUI on dashboard	2 hrs	Multimodal output beyond audio/text

Feature	What It Does	Our Use Case	Effort	Judge Impact
Vertex AI Search	Enterprise search across your own data stores (documents, websites, structured data)	Enable users to upload docs → searchable via Vertex AI Search → agent retrieves relevant passages	4-6 hrs	Deeper than just Google Search grounding

---

Tier 3: Impressive But Niche

Use only if time permits after MVP is solid.

Feature	What It Does	Our Use Case	Effort	Decision
Gemini Thinking	Model shows step-by-step reasoning (chain of thought)	Complex analysis: "Compare these 3 stocks" → model thinks through each step → GenUI shows reasoning chain	1 hr (config)	MAYBE — easy to add
URL Context	Model reads and understands web page content from URLs	"Analyze this article: [URL]" → agent reads page, summarizes, cites sections	1 hr	MAYBE — useful for Chrome extension
Computer Use	Model controls a computer (screenshot → click)	Desktop tray agent: agent sees screen, clicks buttons, fills forms	8+ hrs	STRETCH — aligns with desktop client
Text-to-Speech (TTS)	Dedicated speech synthesis endpoint	Not needed — Live API already handles audio output natively	N/A	SKIP
Speech-to-Text (STT)	Dedicated transcription endpoint	Not needed — Live API's transcription feature handles this	N/A	SKIP
Video Generation (Veo)	Generate video from text/image prompts	Cool but overkill for hackathon	6+ hrs	SKIP
Model Tuning (LoRA/QLoRA)	Fine-tune Gemini on your data	Overkill for 7-day hackathon	10+ hrs	SKIP
Embeddings	Generate vector embeddings for text/images	Only needed if using RAG Engine (it uses embeddings internally)	Included with RAG	AUTO
Agent Garden	Pre-built agent samples and tools from Google	Browse for ideas or tools to integrate	30 min	EXPLORE
Agent Designer	Low-code visual agent builder (Preview)	We're building custom with ADK — not needed	N/A	SKIP
Model Garden	200+ models (Claude, Llama, Mistral, etc.)	Could show multi-model support but complicates hackathon	4+ hrs	SKIP
Example Store	Store/retrieve few-shot examples for agents (Preview)	Improve agent accuracy with dynamic examples	2 hrs	MAYBE

Feature	What It Does	Our Use Case	Effort	Decision
Grounding with Google Image Search	Model retrieves web images as visual context (Preview)	Image-aware answers: "Show me what a monstera plant looks like"	1 hr	MAYBE
Grounding with Vertex AI Search	Ground responses using your own indexed data	Enterprise RAG grounding on top of Vertex AI Search	4+ hrs	STRETCH
Grounding with Elasticsearch	Ground responses using Elasticsearch	Not relevant — we don't use Elasticsearch	N/A	SKIP
Web Grounding for Enterprise	Enterprise-grade web grounding	More control over web grounding than basic Google Search	2 hrs	MAYBE
Inline Citations	Structured `grounding_metadata` linking text to verifiable sources	Already included with Grounding with Google Search — just need to render in GenUI	1 hr	DO — easy win

---

2. Other GCP Services — Beyond Vertex AI

Already Planned

Service	Our Use	Notes
Cloud Run	FastAPI backend hosting with WebSocket support	Set `min_instances=1` to avoid cold start
Firestore	Sessions, personas, MCP configs, chat history	May be partially replaced by Agent Engine Sessions
Firebase Auth	Google sign-in authentication	One-click Google OAuth
Cloud Storage (GCS)	Generated images, code artifacts, file uploads	ADK Artifacts save here
Secret Manager	API keys for MCPs, service account keys	Never hardcode secrets
Artifact Registry	Docker container storage	Cloud Run pulls from here
Cloud Logging	Request/error logging	Show in demo for "production-ready" proof

Worth Adding

Service	What It Does	Our Use Case	Effort	Judge Impact
Cloud Trace	Distributed tracing (OpenTelemetry)	Trace a full request: audio in → ADK processing → tool call → audio out. Show trace viewer in demo	2 hrs	**HIGH** — shows production observability
Cloud Monitoring	Metrics, dashboards, alerting	Dashboard: active sessions, audio latency p99, tool call success rate, errors/min	2 hrs	**HIGH** — "robust hosting" rubric
Cloud CDN + Load Balancing	Global content delivery + traffic distribution	Serve static React assets via CDN. Load balance WebSocket connections	3 hrs	MEDIUM — shows scalability thinking
Pub/Sub	Async message queue	Cross-client event bus: desktop publishes "screenshot taken" → dashboard subscriber shows it. Decouples clients from server	3 hrs	MEDIUM — proper async architecture
Cloud Tasks	Managed task queues	Queue long-running agent jobs (complex research, multi-step TaskArchitect plans)	2 hrs	LOW-MEDIUM
Firebase Hosting	Static web hosting with CDN	Host the React dashboard globally with SSL, free tier	1 hr	LOW — but free and quick
Cloud Build	CI/CD pipeline	Auto-deploy on git push. Show pipeline in demo as "production engineering"	2 hrs	MEDIUM — DevOps maturity
Identity Platform	Advanced auth (MFA, SAML, OIDC)	Overkill — Firebase Auth is sufficient	N/A	SKIP

---

3. Recommended Architecture Stack

Visual Architecture

```
┌──────────────────────────────────────────────────────────┐
│                        VERTEX AI                         │
│                                                          │
│  ┌─────────────────┐  ┌─────────────────┐  ┌──────────────┐ │
│  │ Gemini Live API  │  │ ADK Framework   │  │ Imagen 4     │ │
│  │ (native audio)   │  │ (agents/tools)  │  │ (image gen)  │ │
│  └─────────────────┘  └─────────────────┘  └──────────────┘ │
│          │                    │                   │        │
│  ┌─────────────────┐  ┌─────────────────┐  ┌──────────────┐ │
│  │ Affective Dialog  │  │ Agent Engine    │  │ RAG Engine   │ │
│  │ Proactive Audio   │  │  - Sessions     │  │ (doc Q&A)    │ │
│  │ Barge-in / VAD    │  │  - Memory Bank  │  │              │ │
│  └─────────────────┘  │  - Code Exec    │  └──────────────┘ │
│                       │  - Evaluation   │                   │
│  ┌─────────────────┐  └─────────────────┘                   │
│  │ Grounding         │                                      │
│  │  - Google Search   │  ┌──────────────┐                   │
│  │  - Google Maps     │  │ Cloud Trace  │                   │
│  │  - Image Search    │  │ (OpenTelemetry)                  │
│  └─────────────────┘  └──────────────┘                  │
│                                                          │
├──────────────────────────────────────────────────────────┤
│                     INFRASTRUCTURE                       │
│                                                          │
│  ┌──────────────┐  ┌──────────────┐  ┌─────────────────┐       │
│  │ Cloud Run    │  │ Firestore    │  │ Firebase Auth    │       │
│  │ (backend)    │  │ (data)       │  │ (Google sign-in) │       │
│  └──────────────┘  └──────────────┘  └─────────────────┘       │
│         │               │                  │                 │
│  ┌──────────────┐  ┌──────────────┐  ┌─────────────────┐       │
│  │ Cloud        │  │ Cloud        │  │ Secret Manager   │       │
│  │ Storage      │  │ Monitoring   │  │ (API keys)       │       │
│  │ (GCS)        │  │ + Logging    │  │                  │       │
│  └──────────────┘  └──────────────┘  └─────────────────┘       │
│                                                                 │
│  ┌──────────────┐  ┌──────────────┐  ┌─────────────────┐       │
│  │ Artifact     │  │ Cloud        │  │ Firebase Hosting  │       │
│  │ Registry     │  │ Build        │  │ (static assets)   │       │
│  │ (Docker)     │  │ (CI/CD)      │  │                  │       │
│  └──────────────┘  └──────────────┘  └─────────────────┘       │
│                                                                 │
└──────────────────────────────────────────────────────────┘
```

Service Count Summary

Category	Services	Count
Vertex AI / GenAI	Gemini Live API, ADK, Grounding (Search + Maps), Agent Engine (Sessions + Memory Bank + Code Execution), Gen AI Evaluation, Imagen 4, RAG Engine	**10**
Infrastructure	Cloud Run, Firestore, Firebase Auth, GCS, Secret Manager, Artifact Registry	**6**
Observability	Cloud Logging, Cloud Monitoring, Cloud Trace	**3**
DevOps / Hosting	Cloud Build, Firebase Hosting	**2**
TOTAL		**21**

> **21 Google Cloud services.** No other entry will come close.

4. Feature Deep Dives

4.1 Vertex AI Agent Engine

What it is: A managed runtime that handles the infrastructure for deploying, managing, and scaling AI agents in production. Part of Vertex AI Agent Builder.

Key Services (usable individually or together):

Service	What It Does	GA/Preview
Runtime	Deploy and scale agents with managed infrastructure, VPC-SC, CMEK, IAM	GA
Sessions	Store individual user-agent interactions as conversation context	GA
Memory Bank	Long-term memory — stores/retrieves info across sessions for personalization	GA (uses GenAI models internally)
Code Execution	Run code in secure, isolated, managed sandbox	GA
Example Store	Store/retrieve few-shot examples dynamically	Preview
Evaluation	Assess agent quality with Gen AI Evaluation Service	Preview

Supported Frameworks: ADK (full integration), LangChain (full), LangGraph (full), AG2 (SDK integration), LlamaIndex (SDK integration), CrewAI (custom template), custom frameworks (custom template)

Enterprise Security:

- VPC Service Controls
- Customer-managed encryption keys (CMEK)
- Data residency (DRZ) at rest
- HIPAA compliant
- Access Transparency
- Agent identity via IAM (Preview)
- Threat Detection via Security Command Center (Preview)

Decision for our project:

- **Option A:** Deploy entire backend to Agent Engine Runtime (replaces Cloud Run)
- **Option B:** Keep Cloud Run for custom WebSocket handling + use Agent Engine Sessions, Memory Bank, Code Execution as standalone services
- **Recommendation: Option B** — We need custom WebSocket handling for binary audio frames, which Cloud Run handles better. But use Agent Engine's data services.

Code example (Sessions):

```
from vertexai import Client

client = Client(project="my-project", location="us-central1")

# Create a session
session = client.agent_engine.sessions.create(
    agent_engine_id="my-agent",
    user_id="user-123",
)

# Store interaction
client.agent_engine.sessions.append(
    session_id=session.id,
    messages=[{"role": "user", "content": "What's Tesla stock?"}],
)
```

4.2 Agent Engine Sessions

What it is: Managed stateful session storage that provides definitive sources for conversation context. Replaces custom session persistence logic.

Why it matters for us:

- Our biggest technical risk is session persistence across Cloud Run restarts
- Agent Engine Sessions handles this automatically
- No need for custom Firestore-backed SessionService
- Survives pod restarts, redeployments, scaling events

Integration with ADK:

- ADK's `SessionService` can be backed by Agent Engine Sessions
- Compatible with `SessionResumptionConfig` for WebSocket reconnection

Effort: ~2 hours to integrate **Risk:** LOW — it's GA and well-documented

---

4.3 Agent Engine Memory Bank

What it is: A long-term memory system that stores and retrieves information from past sessions to personalize agent interactions. Goes beyond session-level context.

How it works:

1. After a session ends, Memory Bank uses GenAI models to extract key facts/preferences
2. Facts are stored as structured memories (e.g., "User prefers dark mode", "User works in finance")
3. On new session start, relevant memories are retrieved and injected into agent context
4. Agent appears to "remember" the user across sessions

Why it matters for us:

- **Demo moment:** "Remember last time I asked about Tesla?" → Agent recalls it from Memory Bank, not session history
- **Personalization:** Agent learns user preferences over time
- **No other hackathon entry will have this** — it's a newer service most people don't know about

Effort: ~3 hours (configure Memory Bank, connect to agent, test cross-session recall) **Risk:** MEDIUM — depends on quality of memory extraction

---

4.4 Agent Engine Code Execution

What it is: Managed sandboxed code execution environment. Google-hosted alternative to E2B.

Comparison with E2B:

Aspect	E2B Sandbox	Agent Engine Code Execution
Provider	Third-party (E2B)	Google Cloud (native)
Languages	Python, JS, and many others	Python (primarily)
MCP Gateway	100+ built-in MCPs	No MCP gateway
Pricing	Free tier + pay-per-use	Included with Agent Engine
Reliability	External dependency	Google-managed SLA
Judge Perception	"Uses external service"	"Google Cloud Native"

Recommendation: Use BOTH.

- **E2B** as primary (more languages, MCP gateway)
- **Agent Engine Code Execution** as fallback + for basic Python execution
- Shows judges we maximize Google Cloud AND have redundancy

4.5 Gen AI Evaluation Service

What it is: Enterprise-grade tools for objective, data-driven assessment of generative AI models. Uses adaptive rubrics (like unit tests) to automatically evaluate agent responses.

Key Features:

- **Adaptive Rubrics (Recommended):** Generates unique pass/fail tests per prompt — like auto-generated unit tests
- **Static Rubrics:** Fixed scoring criteria across all prompts (1-5 scale)
- **Computation-based Metrics:** Deterministic algorithms (ROUGE, BLEU) with ground truth
- **Custom Function Metrics:** Define your own Python evaluation logic

How Adaptive Rubrics Work:

```
User Prompt: "Write a four-sentence summary about renewable energy"

Auto-Generated Rubrics:
  ✅ Rubric 1: Response is a summary of the article → PASS
  ✅ Rubric 2: Contains exactly four sentences → PASS
  ❌ Rubric 3: Maintains optimistic tone → FAIL (negative tone in last sentence)

Pass Rate: 66.7%
```

Our Use Case:

1. Create evaluation dataset with 50+ prompts across all 5 personas
2. Test: "Does Nova (analyst) provide accurate financial analysis?"
3. Test: "Does Atlas (coder) write syntactically correct code?"
4. Test: "Does Sage (researcher) cite sources?"
5. Run before submission — include pass rates in blog post
6. Shows judges: "We tested our agent systematically, not just vibes"

Code Example:

```python
from vertexai import Client
from vertexai import types
import pandas as pd

client = Client(project="my-project", location="us-central1")

# Evaluation dataset for Nova persona
prompts_df = pd.DataFrame({
    "prompt": [
        "Analyze Tesla's Q4 2025 earnings report",
        "Compare Bitcoin and Ethereum as investments",
        "What's the current inflation rate in the US?",
    ],
})

# Get responses from the model
eval_dataset = client.evals.run_inference(
    model="gemini-2.5-flash",
    src=prompts_df
)

# Run evaluation with adaptive rubrics
eval_result = client.evals.evaluate(
    dataset=eval_dataset,
    metrics=[types.RubricMetric.GENERAL_QUALITY]
)

eval_result.show()  # Interactive results
```

Supported Regions: us-central1, us-east4, us-west1, us-west4, europe-west1, europe-west4, europe-west9

Effort: 3-4 hours (create dataset, run evals for each persona, document results) **Judge Impact:** VERY HIGH — nobody else will have automated eval results

---

4.6 Grounding with Google Search

What it is: Connects Gemini to Google's search engine to ground responses in real-time web data. Returns inline citations and Search Suggestions.

Key Capabilities:

- Grounds responses in publicly available web data
- Returns `grounding_metadata` with source URLs, inline citations, and search queries
- Supports Google Image Search (Preview) for image-grounded responses
- Search Suggestions must be displayed in production (compliance requirement)
- Limit: 1M queries/day (way more than needed)
- Billing: Per search query generated by Gemini (a single prompt may trigger multiple searches)

What judges see:

- Agent cites sources: "According to Wikipedia [link]..."
- Google Search chip renders below grounded responses
- Factual accuracy is verifiable

Display Requirements (MUST comply):

- Show Search Suggestions exactly as provided (both light and dark mode)
- Search Suggestions must remain visible when grounded response is shown
- Tapping a chip → direct navigation to Google Search results page (no intermediate screens)

Integration with ADK:

```
from google.adk.tools import google_search

# Built-in tool — just add to agent
agent = Agent(
    name="nova",
    tools=[google_search],
    # ...
)
```

4.7 Grounding with Google Maps

What it is: Grounds Gemini responses using Google Maps data for location-based queries.

Our Use Case:

- "Where's the nearest pharmacy?" → grounded location with map data
- Cross-client: glasses user asks for directions → map card renders on web dashboard via GenUI
- Shows another dimension of grounding beyond web search

Effort: 1-2 hours **Documentation:** https://docs.cloud.google.com/vertex-ai/generative-ai/docs/grounding/grounding-with-google-maps

---

4.8 RAG Engine

What it is: Managed vector database + retrieval-augmented generation pipeline. Upload documents, RAG Engine chunks them, creates embeddings, and enables the agent to answer questions about them.

Key Features:

- **RagManagedDb** (default) — fully managed vector store
- Supports Google Drive, GCS, URLs as data sources
- Configurable chunking (chunk size, overlap)
- Reranking for better retrieval quality
- Integrates with Gemini Live API directly
- CMEK + VPC-SC supported

Our Use Case:

- User uploads a PDF/doc → RAG Engine indexes it
- "Summarize this document" → agent retrieves relevant chunks → summarizes
- "What does section 3.2 say?" → precise retrieval

Code Example:

```
from vertexai import rag
from vertexai.generative_models import GenerativeModel, Tool

# Create corpus and import files
rag_corpus = rag.create_corpus(display_name="user_docs")
rag.import_files(rag_corpus.name, ["gs://my_bucket/report.pdf"])

# Create RAG retrieval tool for agent
rag_tool = Tool.from_retrieval(
    retrieval=rag.Retrieval(
        source=rag.VertexRagStore(
            rag_resources=[rag.RagResource(rag_corpus=rag_corpus.name)],
        ),
    )
)

# Use with Gemini
model = GenerativeModel("gemini-2.5-flash", tools=[rag_tool])
response = model.generate_content("Summarize the key findings")
```

Effort: 4-6 hours **Note:** Can be used directly with Gemini Live API — see "Use RAG in Gemini Live API" docs

---

4.9 Imagen 4

What it is: Google's state-of-the-art image generation model. Generate novel images from text prompts at up to 2K resolution.

Key Details:

- Model: `imagen-4.0-generate-001`
- Capabilities: Text-to-image, image editing, image upscaling
- Output: Up to 2K resolution with watermarking
- Alternative: Gemini 3 also has built-in image generation (interleaved text+image output)

When to use Gemini vs Imagen:

Aspect	Gemini	Imagen 4
Strengths	Flexibility, contextual understanding, multi-turn editing	Best quality, fastest latency
Recommended for	Interleaved text+image, conversational editing, combining elements	Photorealism, artistic detail, specific styles, branding, typography
Latency	Higher	Low (near real-time)
Cost	Token-based	Cost-effective for image tasks

Our Use Case:

- Agent tool: `generate_image(prompt)` → returns image → GenUI renders on dashboard
- "Draw a flowchart for this algorithm" → Imagen generates → displayed inline
- Another modality: voice + text + images in one conversation

Code Example:

```
from google import genai
from google.genai.types import GenerateImagesConfig

client = genai.Client()

image = client.models.generate_images(
    model="imagen-4.0-generate-001",
    prompt="A professional flowchart showing a CI/CD pipeline",
    config=GenerateImagesConfig(image_size="2K"),
)

# Save or send to client
image.generated_images[0].image.save("output.png")
```

Effort: 2 hours (create ADK tool wrapper + GenUI image component)

4.10 Gemini Live API

What it is: Low-latency, real-time voice and video interaction with Gemini. The core of our product.

Technical Specifications:

Spec	Value
Input modalities	Audio (16-bit PCM, 16kHz, little-endian), Images/Video (JPEG 1FPS), Text
Output modalities	Audio (16-bit PCM, 24kHz, little-endian), Text
Protocol	Stateful WebSocket (WSS)
Recommended model	`gemini-live-2.5-flash-native-audio`

Key Features:

- High audio quality with natural, realistic speech
- 24 language support with seamless multilingual switching
- Barge-in — users can interrupt anytime
- Affective dialog — adapts tone to user emotion
- Proactive audio (Preview) — model speaks first when context allows
- Tool use — function calling + Google Search during live sessions
- Audio transcriptions — text transcripts for both input and output
- Session resumption — reconnect without losing context

Getting Started Options:

1. **Gen AI SDK tutorial** — Python backend (recommended for ease)
2. **WebSocket tutorial** — JavaScript frontend + Python backend (raw protocol control)
3. **ADK tutorial** — Agent Development Kit streaming (our choice)

Partner Integrations (WebRTC):

- Daily, LiveKit, Twilio, Voximplant
- These handle WebRTC transport — not needed since we use raw WebSocket

---

5. Implementation Priority & Effort Matrix

Phase 1: MVP (Days 1-4)

Priority	Feature	Effort	Status
P0	Gemini Live API + ADK `run_live()`	Core	Must build
P0	Grounding with Google Search	1 hr	Config only
P0	Cloud Run deployment	2 hrs	Standard
P0	Firestore (data persistence)	2 hrs	Standard
P0	Firebase Auth	1 hr	Standard
P0	Cloud Storage (artifacts)	1 hr	Standard
P0	Secret Manager	30 min	Standard

Phase 2: Score Boosters (Days 4-6)

Priority	Feature	Effort	Impact
P1	Agent Engine Sessions	2 hrs	Solves session persistence
P1	Agent Engine Memory Bank	3 hrs	Long-term memory — unique
P1	Gen AI Evaluation Service	3-4 hrs	Production mindset proof
P1	Imagen 4 tool	2 hrs	Image generation modality
P1	Cloud Trace + Monitoring	2 hrs	Observability in demo
P1	Grounding with Google Maps	1-2 hrs	Location intelligence
P2	Agent Engine Code Execution	2 hrs	GCP-native code sandbox

Phase 3: Polish & Bonus (Days 6-7)

Priority	Feature	Effort	Impact
P2	RAG Engine	4-6 hrs	Document Q&A
P2	Cloud Build CI/CD	2 hrs	DevOps maturity
P2	Firebase Hosting (static)	1 hr	CDN for dashboard
P3	Gemini Thinking (config)	1 hr	Chain of thought
P3	URL Context	1 hr	Web page analysis
P3	Inline Citations rendering	1 hr	Visual grounding proof

Total Effort Matrix

```
                        HIGH JUDGE IMPACT
                                |
            ┌──────────────────┼──────────────────┐
            │   QUICK WINS     │   WORTH IT       │
            │                  │                  │
            │ Google Search    │ Agent Engine     │
LOW         │ Grounding (1hr)  │ Sessions (2hrs)  │ HIGH
EFFORT      │ Google Maps      │ Memory Bank      │ EFFORT
            │ Grounding (1hr)  │ (3hrs)           │
            │ Imagen 4 (2hrs)  │ Gen AI Eval      │
            │ Cloud Trace      │ (3-4hrs)         │
            │ (2hrs)           │ RAG Engine       │
            │ Inline Citations │ (4-6hrs)         │
            │ (1hr)            │                  │
            ├──────────────────┼──────────────────┤
            │   EASY ADDS      │   SKIP FOR NOW   │
            │                  │                  │
            │ Cloud Monitoring │ Computer Use     │
            │ (2hrs)           │ (8+hrs)          │
            │ Firebase Hosting │ Video Gen (Veo)  │
            │ (1hr)            │ Model Tuning     │
            │ Cloud Build      │ Vertex AI Search │
            │ (2hrs)           │ (4-6hrs)         │
            │ URL Context      │ Model Garden     │
            │ (1hr)            │                  │
            └──────────────────┼──────────────────┘
                        LOW JUDGE IMPACT
```

Execution order: Top-left → Top-right → Bottom-left. Skip bottom-right.

6. GCP Service Count — Competitive Analysis

Our Project: 16-21 Services

#	Service	Category
1	Gemini Live API	Vertex AI
2	ADK (Agent Development Kit)	Vertex AI
3	Grounding w/ Google Search	Vertex AI
4	Grounding w/ Google Maps	Vertex AI
5	Agent Engine Sessions	Vertex AI
6	Agent Engine Memory Bank	Vertex AI
7	Agent Engine Code Execution	Vertex AI
8	Gen AI Evaluation Service	Vertex AI
9	Imagen 4	Vertex AI
10	RAG Engine	Vertex AI (stretch)
11	Cloud Run	Infrastructure
12	Firestore	Infrastructure
13	Firebase Auth	Infrastructure
14	Cloud Storage (GCS)	Infrastructure
15	Secret Manager	Infrastructure
16	Artifact Registry	Infrastructure
17	Cloud Logging	Observability
18	Cloud Monitoring	Observability
19	Cloud Trace	Observability
20	Cloud Build	DevOps
21	Firebase Hosting	DevOps

Typical Competitor: 4-6 Services

#	Service	Usage
1	Gemini API	Text/audio generation
2	ADK	Basic agent
3	Cloud Run	Hosting
4	Firestore	Storage
5	Firebase Auth	Login
6	GCS	Files

Our Advantage

```
Us:    ██████████████████ 21 services
Them:  ██████ 6 services

"Google Cloud Native" score advantage: MASSIVE
```

This depth of Google Cloud usage — spanning Vertex AI GenAI, Agent Engine data services, infrastructure, observability, and DevOps — is virtually impossible for competitors to match in a hackathon timeframe. Most entries will use Gemini + ADK + Cloud Run and call it done.

Sources

- Vertex AI Generative AI Overview: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/overview
- Vertex AI Agent Builder Overview: https://docs.cloud.google.com/agent-builder/overview
- Vertex AI Agent Engine Overview: https://docs.cloud.google.com/agent-builder/agent-engine/overview
- Gemini Live API Overview: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/live-api
- Grounding with Google Search: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/grounding/grounding-with-google-search
- Gen AI Evaluation Service: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/evaluation-overview
- RAG Engine Quickstart: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/rag-quickstart
- Imagen on Vertex AI: https://docs.cloud.google.com/vertex-ai/generative-ai/docs/image/overview